

Activity 12: Integrated incident and change verification

TGS-2023037854 | v7.0 | 13 September 2026 | 70 minutes

Goal

Use time-correlated service evidence to isolate a trunk fault and verify a narrow change with regression tests and rollback.

Before you start

Python 3.9+, a plain text editor and this complete folder. Offline mode requires no extra packages, Internet or credentials. The compulsory core Packet Tracer construction is specified below and in topology.md; advanced extensions may need CML/IOS XE.

Evidence boundary

All JSON/CSV observations are synthetic training evidence. Passing the Python checker validates the fixture contracts only. No hardware, simulator, authentication server or API is configured or contacted.

Files

README.md, instructions.pdf, checklist.pdf, topology.md, case.json, faulty.json, reference.json, observations.csv, specimen.cfg, S1-baseline.cfg, S2-baseline.cfg, D1-baseline.cfg

Working method

case.json is the working evidence; faulty.json resets the exercise and reference.json shows corrected synthetic evidence for self-study. Preserve your own sandbox observations separately.

Detailed step-by-step instructions

1. Construct and save the mandatory simulator topology

Start Cisco Packet Tracer 8.x or a current compatible version. File > New; use the bottom device palette, drag these models into the workspace and rename them via Config > Display Name: 1 x Multilayer Switch3560-24PS (D1), 2 x Switch2960 (S1,S2), 1 x PC-PT (PC1), 1 x Server-PT (APP), 1 x PC-PT (MGMT). Choose Connections > Copper Straight-Through for the table unless otherwise stated. Switch-to-switch links use Copper Cross-Over or Automatically Choose Connection Type. Click the first device and select its named interface; click the peer and select its named interface. Wait for links to initialize. Save as activity-start.pkt.

Device/port A	Device/port B	Cable
PC1 FastEthernet0	S1 FastEthernet0/1	Straight-through
APP FastEthernet0	S2 FastEthernet0/1	Straight-through
S1 GigabitEthernet0/1	D1 GigabitEthernet0/1	Cross-over/Auto
S2 GigabitEthernet0/1	D1 GigabitEthernet0/2	Cross-over/Auto
MGMT FastEthernet0	D1 FastEthernet0/1	Straight-through

Save both activity-start.pkt and a working activity-working.pkt; use File > Save As.

2. Assign endpoint addresses and inspect interface identifiers

Set Desktop > IP Configuration > Static on each endpoint.

Endpoint	Address	Mask	Gateway	DNS
PC1	10.10.10.10	255.255.255.0	10.10.10.1	10.10.20.20
APP	10.10.20.20	255.255.255.0	10.10.20.1	10.10.20.20
MGMT	198.51.100.10	255.255.255.0	198.51.100.1	none

APP Services > HTTP On (HTTPS On if available); DNS On with app.example.test A10.10.20.20. PC1 has APP as resolver. The unaffected ping target is MGMT198.51.100.10, NOT the APP server on the missing VLAN. For routers/switches open CLI. Answer no to the initial configuration dialog and press Enter. Use the exact interface names below; a 2911 uses GigabitEthernet0/0 through 0/2, a 2960/3560 uses FastEthernet0/1 through 0/24 and GigabitEthernet0/1 through 0/2.

```
enable
show ip interface brief
```

3. Install all peer baseline configurations

Paste each named baseline, including the MGMT-ISOLATE ACL on Vlan40; preserve this policy during the trunk repair.

```
Device S1
enable
configure terminal
hostname S1
no ip domain-lookup
vlan 10
    name USERS
vlan 20
    name SERVICES
vlan 99
    name NATIVE
interface FastEthernet0/1
```

```

switchport mode access
switchport access vlan 10
no shutdown
exit
interface GigabitEthernet0/1
switchport mode trunk
switchport trunk native vlan 99
switchport trunk allowed vlan 10,20,99
no shutdown
exit
end
copy running-config startup-config

```

Device S2

```

enable
configure terminal
hostname S2
no ip domain-lookup
vlan 10
name USERS
vlan 20
name SERVICES
vlan 99
name NATIVE
interface FastEthernet0/1
switchport mode access
switchport access vlan 20
no shutdown
exit
interface GigabitEthernet0/1
switchport mode trunk
switchport trunk native vlan 99
switchport trunk allowed vlan 10,20,99
no shutdown
exit
end
copy running-config startup-config

```

Device D1

```

enable
configure terminal
hostname D1
no ip domain-lookup
vlan 10
name USERS
vlan 20
name SERVICES
vlan 99
name NATIVE
interface range GigabitEthernet0/1-2
switchport trunk encapsulation dot1q
switchport mode trunk
switchport trunk native vlan 99
switchport trunk allowed vlan 10,20,99
no shutdown
exit
ip routing
interface Vlan10
ip address 10.10.10.1 255.255.255.0
no shutdown
exit
interface Vlan20
ip address 10.10.20.1 255.255.255.0
no shutdown
exit
vlan 40
name MANAGEMENT-TEST

```

```

interface FastEthernet0/1
  switchport mode access
  switchport access vlan 40
  no shutdown
exit
interface Vlan40
  ip address 198.51.100.1 255.255.255.0
  no shutdown
exit
ip access-list extended MGMT-ISOLATE
  deny ip 198.51.100.0 0.0.0.255 10.10.20.0 0.0.0.255
  permit ip any any
exit
interface Vlan40
  ip access-group MGMT-ISOLATE in
exit
end
copy running-config startup-config

show running-config
show ip interface brief
copy running-config startup-config

```

4. Prove the healthy baseline with actual packets

Run every test below before introducing the fault. PC/Desktop > Command Prompt provides ping, ipconfig and arp. Use Simulation mode and Edit Filters to keep ARP, ICMP and the protocol under study; click Add Simple PDU for a ping and Capture/Forward to inspect hops. The first ping may lose one packet during ARP; repeat and record the second result. Do not mark a test passed until observed.

Baseline/repaired PC1 ping 10.10.10.1, ping 198.51.100.10 and ping 10.10.20.20 succeed. PC1 Web Browser <http://app.example.test> displays APP page; HTTPS optional according to PT support. During fault only the unaffected gateway/MGMT tests succeed. S2 show interfaces trunk reveals missing20. D1 show ip interface brief and show ip route connected distinguish SVI state from trunk allowance. Record pre/post APP service and management reachability; a successful management ping is not APP reachability.

Mandatory denied regression: MGMT ping 10.10.20.20 fails because MGMT-ISOLATE denies VLAN40 to APP VLAN20. PC1 ping APP succeeds. show access-lists MGMT-ISOLATE shows the deny hit counter. Repeat these same allowed/denied tests after trunk repair without modifying the ACL.

After all healthy tests pass, File > Save As > activity-start.pkt to preserve the CONFIGURED healthy baseline, then File > Save As > activity-working.pkt before introducing the fault. The initial empty construction is not the rollback baseline.

Save baseline screenshots / CLI text with device names; File > Save.

5. Introduce and diagnose the controlled fault

Save the baseline first. Apply only the following fault to the specified device. Repeat the same traffic tests, capture the first failing hop/control and compare show output with the healthy baseline.

On S2 Gi0/1 set allowed VLAN10,99, removing20. PC1 gateway10.10.10.1 and MGMT198.51.100.10 stay reachable, but APP10.10.20.20, DNS at APP and application traffic fail. First diagnose the trunk from actual evidence; do not weaken an ACL to repair it. The offline fixture DNS_ok true represents an independent resolver/cached result and is not proof the affected APP resolver remains live.

Retain activity-start.pkt and baseline running configurations; work in activity-working.pkt.

6. Apply the narrow repair and re-test the network

Use S2 configure terminal; interface GigabitEthernet0/1; switchport trunk allowed vlan add 20 (space: add 20); end. Restore the exact saved native/allowed lists if any other value was changed. No ACL modification is authorized or required in this scenario.

Repeat the baseline test matrix. Check surrounding controls as well as the repaired flow. how a narrow VLAN repair restores the affected service while preserving independent management paths and surrounding policy

Save actual show output and allowed/denied traffic results; copy running-config startup-config only after successful verification.

7. Rollback and rehearse recovery

Save the repaired file as activity-verified.pkt. To undo the exercise, open the saved healthy baseline activity-start.pkt after you have saved configurations into it at Step 3, or paste the saved pre-fault running configuration into a clean topology. For the specific repair, retain the exact old command/value rather than guessing a trunk list or ACL. Repeat one positive and one relevant negative test after recovery.

File > Save As > activity-verified.pkt; close working topology; reopen saved healthy baseline.

8. Submit observed evidence and explain the forwarding mechanism

Submit the verified .pkt file, a cabling/address table, pre-fault and post-repair show outputs, screenshots of traffic tests, diagnosis and rollback notes. Explain how a narrow VLAN repair restores the affected service while preserving independent management paths and surrounding policy Keep synthetic checker output separate from real simulator observations.

Record simulator version, device models and tests actually completed.

9. Complete the separately labelled extension

Preserve the baseline MGMT-ISOLATE ACL. Repeat MGMT-to-APP denial and PC1-to-APP success after the trunk repair; inspect ACL counters. HTTPS depends on Server-PT support; record HTTP as observed application traffic and HTTPS unavailable if absent. The offline dns_ok result is an independent resolver result; the simulation resolver on APP becomes unreachable during the VLAN fault. Document this distinction rather than reporting a DNS success you did not observe.

Record supported platform, output observed or evidence-only limitation.

Untimed appendix: independent fixture analysis

These synthetic cases supplement the allocated hands-on time and do not configure the simulator.

Appendix 1. Establish the baseline

Open activity12-integrated-incident-change-verification; inspect topology.md, case.json and observations.csv. Identify the controls for: Use time-correlated service evidence to isolate a trunk fault and verify a narrow change with regression tests and rollback.

```
python3 --version
```

Appendix 2. Run the faulty evidence case

Run the checker against case.json; exit status 1 is expected at this stage. Record each FAIL label and the calculated detail.

```
python3 checker.py case.json
```

Appendix 3. Propose the repair

Set after `acl_expected true`; restore target trunk allowed VLAN20, scope [trunk VLAN20], approval trainer sandbox, rollback restore saved trunk allowed VLAN list and `saved_before true`. Reject the broad ACL change even if HTTPS appears restored.

Open `case.json` in a plain text editor; preserve JSON types.

Appendix 4. Validate the corrected case

Save a separate working copy first. Run the checker after each narrow edit until all controls PASS; capture output as evidence. This validates fixtures only.

```
python3 checker.py case.json > evidence.txt
```

Appendix 5. Verify checker behavior

Run self-test; it must accept corrected reference evidence and reject the faulty fixture and the independent corrupted-control test.

```
python3 checker.py --self-test
```

Expected results and evidence

Corrected reference: all checks PASS, exit 0; faulty fixture: one or more FAIL, exit 1; self-test: PASS, exit 0.

Domain-specific computed fields appear in JSON detail.

Capture `evidence.txt`, corrected `case.json` and your diagnosis. For sandbox work record device/model, timestamps, actual show outputs, allowed/denied tests and rollback. Distinguish fixture results from real observations.

Troubleshooting and reset

Exit 1: inspect the named failed contract. Exit 2: restore valid JSON keys/types. Unsupported simulator commands must be documented; never invent results. Reset by copying `faulty.json` over `case.json`. Restore sandbox settings from the actual saved baseline.

Calculated corrected-fixture results

`restored_HTTPS: true`

`change_scope: ["trunk VLAN20"]`

`verification_seconds: 40`

Required PASS controls

- isolated trunk hypothesis
- repaired services and preserved ACL
- narrow approved reversible change
- regressions include isolation
- ordered approved timeline

Configuration files

The complete per-device configurations are shown in the construction steps and included as named `*-baseline.cfg` files. `specimen.cfg` combines those same blocks with explicit device boundaries. Apply each block only to its named device. WRT300N uses GUI settings instead of IOS CLI.

References

Cisco CCNA v1.1 exam topics; Cisco IOS XE configuration guides; Python `ipaddress` documentation. Exact URLs and source links are in `README.md`.